

MongoDB CRUD



Objectives

After completing this lab, you will be able to:

- Create documents in MongoDB with the insert method
- Read documents by listing them, counting them and matching them to a query
- Update and delete documents in MongoDB based on specific criteria

Exercise 1 - Getting the environment ready

Open the MongoDB database page by clicking the button below:

 Open MongoDB Page in IDE

On that page, click the **Create** button to create a MongoDB database.

After creation has finished, click the **MongoDB CLI** button (below the **Create** button from the previous step) to connect to the database using the mongosh CLI.

1. Problem:

Select the *training* database.

▼ Click here for Hint

Refer to the *Mongodb Getting Started lab*.

▼ Click here for Solution

1 `use training`



SKILLS NET...

> DATABASES

> BIG DATA

> CLOUD

> EMBEDDABLE AI

> OTHER

Launch Applica...

Welcome

MongoDB

MySQL

ACTIVE

3.6.3

0.54.0

2.0.2

Connect to MongoDB and mongo-express directly in your Skills Network Labs environment.

CreateDelete

Summary

Connection Information

Details

Your database and mongo-express server are now ready to use and available with the following login credentials. For more details on how to navigate MongoDB, please check out the Details section.

You can manage MongoDB via:

mongo-express

Or to interact with the database in the terminal, select one of these options:

MongoDB CLI

New Terminal

eia@theiadocker-naimbenalaya: /home/project

mongosh mongod

2025-07-09T17:20:56.008+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See <http://dochub.mongodb.org/core/prodnotes-filesystem>

test> use training
switched to db training
training>

2. Problem:

Create a collection named *languages*.

▼ Click here for Hint

Refer to the [MongoDb Getting Started lab](#).

▼ Click here for Solution

1

```
db.createCollection("languages")
```

MongoDB CRUD

2

Exercise 2 - Insert documents

Let us insert five documents into the collection **languages**

On the mongo client run the below commands.

```
1 db.languages.insert({"name":"java","type":"object oriented"})
2 db.languages.insert({"name":"python","type":"general purpose"})
3 db.languages.insert({"name":"scala","type":"functional"})
4 db.languages.insert({"name":"c","type":"procedural"})
5 db.languages.insert({"name":"c++","type":"object oriented"})
```

```
training> db.languages.insert({"name":"java","type":"object oriented"})
... db.languages.insert({"name":"python","type":"general purpose"})
... db.languages.insert({"name":"scala","type":"functional"})
... db.languages.insert({"name":"c","type":"procedural"})
... db.languages.insert({"name":"c++","type":"object oriented"})
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('686ed9baf082bb87246b140f') }
}
```

Exercise 3 - Read documents

Let us try out different ways of querying documents.

Find the count of documents.

```
1 db.languages.countDocuments()
```

List the first document in the collection.

```
1 db.languages.findOne()
```

List all documents in the collection.

```
1 db.languages.find()
```

```
ieia@theiadocker-naimbenalaya: /home/project mongosh mongodb://<credentials>@172.21.51.59:27017/?directConnection=true X [ ]
training> db.languages.countDocuments()
5
training> db.languages.findOne()
{
  _id: ObjectId('686ed9baf082bb87246b140b'),
  name: 'java',
  type: 'object oriented'
}
training> db.languages.find()
[
  {
    _id: ObjectId('686ed9baf082bb87246b140b'),
    name: 'java',
    type: 'object oriented'
  },
  {
    _id: ObjectId('686ed9baf082bb87246b140c'),
    name: 'python',
    type: 'general purpose'
  },
  {
    _id: ObjectId('686ed9baf082bb87246b140d'),
    name: 'scala',
    type: 'functional'
  },
  {
    _id: ObjectId('686ed9baf082bb87246b140e'),
    name: 'c',
    type: 'procedural'
  }
]
```

List first 3 documents in the collection.

1	<code>db.languages.find().limit(3)</code>	 
---	---	---

```
]
training> db.languages.find().limit(3)
[
  {
    _id: ObjectId('686ed9baf082bb87246b140b'),
    name: 'java',
    type: 'object oriented'
  },
  {
    _id: ObjectId('686ed9baf082bb87246b140c'),
    name: 'python',
    type: 'general purpose'
  },
  {
    _id: ObjectId('686ed9baf082bb87246b140d'),
    name: 'scala',
    type: 'functional'
  }
]
training> █
```

Query for "python" language.

```
1 db.languages.find({"name":"python"})
```

Query for "object oriented" languages.

```
1 db.languages.find({"type":"object oriented"})
```

```
training> db.languages.find({"name":"python"})
[
  {
    _id: ObjectId('686ed9baf082bb87246b140c'),
    name: 'python',
    type: 'general purpose'
  }
]
training> db.languages.find({"type":"object oriented"})
[
  {
    _id: ObjectId('686ed9baf082bb87246b140b'),
    name: 'java',
    type: 'object oriented'
  },
  {
    _id: ObjectId('686ed9baf082bb87246b140f'),
    name: 'c++',
    type: 'object oriented'
  }
]
training> 
```

List only specific fields.

Using a projection document you can specify what fields we wish to see or skip in the output.

This command lists all the documents with only name field in the output.

```
1 db.languages.find({}, {"name":1})
```

This command lists all the documents without the name field in the output.

```
1 db.languages.find({}, {"name":0})
```

This command lists all the "object oriented" languages with only "name" field in the output.

```
1 db.languages.find({"type":"object oriented"}, {"name":1})
```

```

]
training> db.languages.find({}, {"name":1})
[
  { _id: ObjectId('686ed9baf082bb87246b140b'), name: 'java' },
  { _id: ObjectId('686ed9baf082bb87246b140c'), name: 'python' },
  { _id: ObjectId('686ed9baf082bb87246b140d'), name: 'scala' },
  { _id: ObjectId('686ed9baf082bb87246b140e'), name: 'c' },
  { _id: ObjectId('686ed9baf082bb87246b140f'), name: 'c++' }
]
training> db.languages.find({}, {"name":0})
[
  {
    _id: ObjectId('686ed9baf082bb87246b140b'),
    type: 'object oriented'
  },
  {
    _id: ObjectId('686ed9baf082bb87246b140c'),
    type: 'general purpose'
  },
  { _id: ObjectId('686ed9baf082bb87246b140d'), type: 'functional' },
  { _id: ObjectId('686ed9baf082bb87246b140e'), type: 'procedural' },
  {
    _id: ObjectId('686ed9baf082bb87246b140f'),
    type: 'object oriented'
  }
]
training>

```

```

training> db.languages.find({"type":"object oriented"}, {"name":1})
[
  { _id: ObjectId('686ed9baf082bb87246b140b'), name: 'java' },
  { _id: ObjectId('686ed9baf082bb87246b140f'), name: 'c++' }
]
training>

```

Exercise 4 - Update documents

Update documents based on a criteria.

Add a field to all the documents.

The 'updateMany' command is used to update documents in a mongodb collection, and it has the following generic syntax.

```
1 db.collection.updateMany({what documents to find},{set:{what fields to set}})
```

Here we are adding a field **description** with value **programming language** to all the documents.

```
1 db.languages.updateMany({},{$set:{description:"programming language"}})
```

Set the creator for python language.

```
1 db.languages.updateMany({"name":"python"},{$set:{creator:"Guido van Rossum"}})
```

Set a field named **compiled** with a value **true** for all the object oriented languages.

```
1 db.languages.updateMany({"type":"object oriented"},{$set:{compiled:true}})
```

```
training> db.languages.updateMany({},{$set:{description:"programming language"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 5,
  modifiedCount: 5,
  upsertedCount: 0
}
training> db.languages.updateMany({"name":"python"},{$set:{creator:"Guido van Rossum"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
training> db.languages.updateMany({"type":"object oriented"},{$set:{compiled:true}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
training> 
```

Exercise 5 - Delete documents

Delete documents based on a criteria.

Delete the scala language document.

```
1 db.languages.remove({"name":"scala"})
```

Delete the object oriented languages.

```
1 db.languages.remove({"type":"object oriented"})
```

Delete all the documents in a collection.

```
1 db.languages.remove({})
```

```
training> db.languages.remove({"name":"scala"})
DeprecationWarning: Collection.remove() is deprecated. Use deleteOne, deleteMany, findOneAndDelete, or bulkWrite.
{ acknowledged: true, deletedCount: 1 }
training> db.languages.remove({"type":"object oriented"})
{ acknowledged: true, deletedCount: 2 }
training> db.languages.remove({})
{ acknowledged: true, deletedCount: 2 }
training>
```

Go Live 1